

Procesos y threads

Un *thread* (hilo) es la secuencia de instrucciones que la CPU está ejecutando o ejecutará. El kernel *multithreaded* crea varios hilos de ejecución, generalmente para realizar ciertas tareas que le podrían llevar mucho tiempo para ejecutarlos (*planificarlos*) cuando la CPU esté *idle* (no haya un procesos de usuario listo para asignarle la CPU).

Un *proceso* tiene al menos un hilo de ejecución y se corresponde con una *aplicación* o programa de usuario. En particular un thread de un proceso se ejecutará en *modo usuario* (el menor modo de privilegio de la CPU).

Threads

Cada proceso (y el kernel) tienen al menos un *hilo* o *thread* de ejecución.

El estado de un *thread* en un punto dado de su ejecución está representado por:

- El *program counter* (*pc*): Dirección de la próxima instrucción a ejecutar.
- El *stack pointer* (*sp*): Dirección del *tope* de la pila.
- El *frame (o base) pointer* (*fp*): Dirección base del registro de activación del tope.

La pila contiene *registros de activación* para el control de la secuencia de invocaciones a funciones y sus retornos. Además comúnmente contienen los *datos automáticos* (variables locales y argumentos) de las funciones activas. De ese modo se crean y eliminan automáticamente los *ambientes locales*.

La representación de un *registro de activación* se basa en el *calling convention* de la *ABI* de la plataforma.

El *frame pointer* se usa en las instrucciones del programa como dirección base para acceder en forma relativa (desplazamientos constantes) a las variables locales y argumentos dentro del registro de activación.

Los pasos en una invocación a una función *f* son los siguientes:

1. Invocante: Reserva espacio para el valor de retorno, si es que se dejará en la pila. Si se retorna en un registro, este paso se omite.
2. Invocante: Apila (o pone en registros) los valores de los argumentos.
- 3.

Invocante: Ejecuta la instrucción `call f`. En algunas arquitecturas ésta instrucción *apila* el valor del *program counter* y luego le asigna la dirección de la primera instrucción de `f`.

4. En alguna<s arquitecturas (como RISC-V), el compilador generó instrucciones para *apilar* la dirección de retorno si `call` no lo hace.
5. `f` *apila* (salva) el valor el *frame pointer* y le asigna el valor del *stack pointer*.
6. `f` posiblemente reserva espacio para las *variables locales*, restando un valor al *stack pointer*. Estos pasos (4-6) se conocen como el *prólogo*.
7. `f` ejecuta las instrucciones del *cuerpo* de `f`.
8. `f` comienza el retorno (*epílogo*). Se *eliminan las variables locales y temporarios* (`sp=fp`).
9. `f` recupera el *base pointer* salvado (`pop fp`).
10. `f` retorna (ejecuta la instrucción `ret`). En algunas arquitecturas, como X86, `ret` espera la dirección de retorno en el tope del stack y lo desapila. En otras, como RISC-V `ret` copia en el `pc` el valor de `ra`. En el caso de haberse apilado, el compilador habrá generado instrucciones de *desapilado* en `ra` previo a `ret`.
11. Invocante: Desapila los argumentos, si hubiera, y toma el valor retornado (en la pila o desde registros de la CPU).

Esta secuencia de operaciones deja un *activation record* con el siguiente formato:

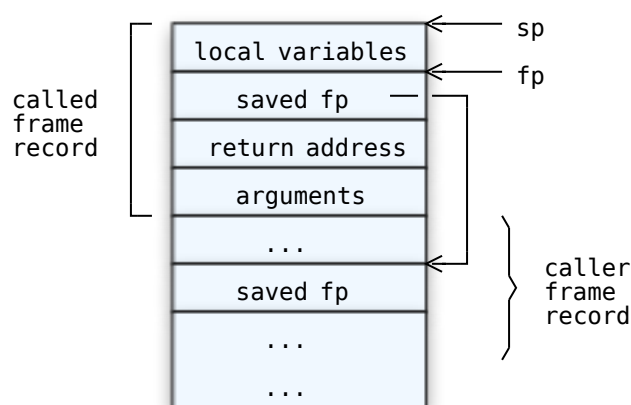


Figura 1: Pila de registros de activación de funciones.

El *calling convention* en una ABI de un SO o plataforma de hardware especifica los detalles de estos pasos. En particular, cómo se pasan los argumentos (en registros, pila o ambos) y el valor retornado (en registros o en la pila). Por ejemplo, Linux para x86-64 usa la especificación [System V ABI](#).

RISC-V

Una cpu *risc-v* tiene 32 registros de propósitos generales más el *program counter* (*pc*). Algunos de sus registros se usan según la siguiente convención:

- *x1* o *ra* : Return address (set by *call* or *jmp* instructions)
- *x2* o *sp* : Stack pointer.
- *x8* o *fp* : Frame pointer.
- *x10-x17* (*a0-a7*): Argumentos. En caso de funciones con más de 8 argumentos se deben apilar en el stack.
- El valor de retorno debe quedar en *a0* (y *a1* , opcionalmente). Los
- Los registros *s0-s11* deben ser salvados (en la pila) por la función invocada.

En el kernel del SO existe un thread por CPU que luego de la inicialización del sistema estarán ejecutando el cuerpo de *scheduler()* (en un ciclo infinito).

Un kernel de un SO podría ser *multithreaded*.

Creación de threads y Cambios de contexto

Para crear un nuevo thread se debe crear el espacio de memoria para su pila. En la pila se crea el *contexto inicial* que consiste en los valores de los registros de la CPU que deberán usarse al inicio del thread. En particular, el valor del *program counter* (*pc*), que deberá apuntar a la primera instrucción a ejecutar, en general el inicio de una función.

La función de creación de un thread puede ser de la forma:

```
void create_thread(void(*pc)(void), uint* stack_ptr)
{
    // push callee saved register values on stack
    push(&stack_ptr, 0); // s0 in RISC-V
    // ...
    push(&stack_ptr, 0); // s11 in RISC-V
    // push the first instruction address to execute
    push(&stack_ptr, pc);
}
```

La función de cambio de contexto comúnmente tiene la forma

switch_context(¤t_ctx, &new_ctx) y deberá realizar los siguientes pasos:

1. Salvar (apilar) los valores de los registros de la CPU, en particular la dirección de retorno (*ra* en RISC-V), en la pila corriente (apuntada por *sp*) y actualizar

- `current_ctx` al nuevo valor de `sp` .
2. Cambiar al stack del nuevo thread: `sp <-- *new_ctx` .
3. Restaurar (pop) los valores salvados de los registros de la CPU (incluyendo el `pc`) desde el tope de la pila apuntado por `new_ctx` y actualizar `new_context` .

Abajo se muestra una implementación en assembly para RV32 (RISC-V 32 bits).

c

```
# context_switch(uint* old_sp, uint* new_sp)
# a0=old_sp, a1=new_sp
context_switch:
    # push callee saved registers and task pc (ra)
    addi sp, sp, -13 * 4
    sw ra, 0 * 4(sp) # push return address
    sw s0, 1 * 4(sp) # push callee saved registers
    sw s1, 2 * 4(sp)
    sw s2, 3 * 4(sp)
    sw s3, 4 * 4(sp)
    sw s4, 5 * 4(sp)
    sw s5, 6 * 4(sp)
    sw s6, 7 * 4(sp)
    sw s7, 8 * 4(sp)
    sw s8, 9 * 4(sp)
    sw s9, 10 * 4(sp)
    sw s10, 11 * 4(sp)
    sw s11, 12 * 4(sp)

    # update old_sp and set sp to new_sp
    sw sp, (a0)      # *old_sp <- sp;
    lw sp, (a1)      # sp <- *new_sp

    # restore register values from new stack
    lw ra, 0 * 4(sp) # restore return address
    lw s0, 1 * 4(sp) # restore callee saved registers
    lw s1, 2 * 4(sp)
    lw s2, 3 * 4(sp)
    lw s3, 4 * 4(sp)
    lw s4, 5 * 4(sp)
    lw s5, 6 * 4(sp)
    lw s6, 7 * 4(sp)
    lw s7, 8 * 4(sp)
    lw s8, 9 * 4(sp)
    lw s9, 10 * 4(sp)
    lw s10, 11 * 4(sp)
    lw s11, 12 * 4(sp)
    addi sp, sp, 13 * 4 # simulate pop
```

ret

Se debe notar que el primer parámetro es de *salida* (se actualiza al nuevo tope del stack del *current_thread*). El parámetro *new_ctx* es por referencia, ya que debe actualizarse a medida que se van desapilando los valores del stack del nuevo thread en los registros de la CPU.

A continuación se muestra un fragmento de código sobre el uso de estas funciones.

c

```
// a caller thread creating a new thread starting in function f()
uint* main_thread_sp;
unsigned char new_thread_stack[STACK_SIZE];
uint* new_thread_sp = (uint*) (new_thread_stack + STACK_SIZE);

void f(void)
{
    // ...
    // resume execution of main thread
    context_switch(&new_thread_sp, &main_thread_sp);
    // ...
}

int main(void)
{
    create_thread(f, &new_thread_sp);
    // start new thread: run f() body
    context_switch(&main_thread_sp, &new_thread_sp);
    // main thread resume
    // ...
    // resume f thread again
    context_switch(&main_thread_sp, &new_thread_sp);
}
```

Estos cambios de contexto son *explícitos*. Cada thread especifica a qué otro thread desea darle el control de la CPU. En éste caso se denominan *corrutinas*.

Comúnmente, en un SO, un thread al abandonar la CPU usará una función *yield()* que transfiere el control al thread principal que estará en *scheduler()*. Esta última función decide cuál será la próxima tarea a la que se le dará el control de la CPU.

Las funciones *yield()* y *scheduler()* tienen la siguiente forma:

```

struct task* current; // current task descriptor
...
void yield()
{
    current->state = RUNNABLE;
    switch_context(&current->stack, &scheduler_ctx);    -----+
}                                                         |
                                                         |
void scheduler() {                                       |
    ...                                                 |
    while (1) {                                         |
        current = NULL;                                |
        ...                                             |
        struct task* = select_runnable_task();         |
        if (task) {                                    |
            context_switch(&scheduler_ctx, &(task->sp)); |
        }                                               <-----+
    }
    ...
}
}

```

En el caso que los threads invoquen explícitamente a `yield()` se dice que el sistema soporta **multitarea cooperativa** ya que cada thread *colabora* cediendo la CPU periódicamente

Un thread que no invoque a `yield()` monopolizará la CPU. Para prevenir esto, principalmente para la implementación de procesos de usuario, el kernel usa el *timer* (*reloj*) que genera interrupciones periódicamente. En una interrupción del timer, el kernel le puede *quitar* la CPU al thread corriente (invocando a `yield()`). Esto se conoce como **preemptive multitasking**.

edos-steps

En la carpeta `05-tasks` se muestra la implementación de estas funciones en el kernel y el uso de *kernel threads* (o *kernel tasks*) cooperativos. En el paso `06-preemptive` se implementa *preemptive multitasking*.

Procesos de usuario

Un *proceso* representa un *programa de usuario en ejecución*. Generalmente se representa con al menos los siguientes atributos:

- Un identificador (*process id* o *pid*).
- Su estado: Ver la [figura 3](#).
- Razón o recurso por el cual está en estado **SLEEPING** .
- Su *mapa* (áreas) de memoria a las cuales puede acceder en modo usuario.
 - Código (*text segment*).
 - Datos globales (*estático*): Área de datos inicializados (*data section*) y no inicializados (*bss section*).
- Al menos un *thread* de ejecución, representado por
 - Stack en modo usuario. Stack que usa el *thread* mientras ejecuta en modo usuario.
 - Stack en modo kernel. En este stack se almacena el *trapframe*: Valores de los registros de la CPU al ocurrir una interrupción en éste *thread*. Esta pila también se usa para el control de invocación a funciones dentro del kernel ejecutándose en el *contexto del proceso*. Finalmente, también se usa para guardar el *contexto* (*caller saved CPU registers*) del *thread* cuando se produzca un *context switch* al scheduler.
- Conjuntos de recursos adquiridos: Archivos/pipes/sockets abiertos, semáforos, timers, áreas de memoria compartida, manejadores de señales y otros.
- El *comando* o *programa* del cual es una instancia.

Linux

GNU-Linux representa a un proceso por medio de una `struct task_struct` , el cual contiene los atributos básicos del *thread* y punteros a estructuras de datos que representan los demás recursos (archivos, memoria, etc). En un proceso *multithread* cada `struct task_struct` apuntan a los mismos recursos, ya que pertenecen al mismo proceso y tienen el mismo *thread group=PID* aunque diferente *thread id*.

Cuando un proceso se carga en memoria, mediante el syscall `exec(path, args)` , sus secciones de código (*text*) y datos (*data*) se cargan desde el archivo ejecutable (en formato ELF, por ejemplo), se le asigna espacio para el stack del *thread* principal y se inicializa un *stackframe* conteniendo los valores de los registros de la CPU requeridos al inicio de la ejecución. En particular los valores que deberán tomar el *program counter* (*pc*) y el *stack pointer* (*sp*).

El valor inicial del `sp` deberá apuntar a la primera instrucción de la función de inicio del proceso, comúnmente la función `_start` de la biblioteca estándar (enlazada con cada programa de usuario). Esta función comúnmente se define como:

c

```
void _start(void)
{
    exit(main());
}
```

El stack se inicializa de tal forma que `main(int argc, char* argv[])` reciba los parámetros que correspondan.

La siguiente figura muestra el esquema (*layout*) en memoria de un proceso.

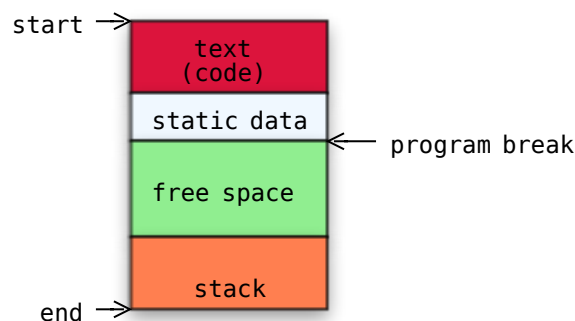


Figura 2: *Layout (virtual) de memoria de un proceso.*

El **espacio de direcciones** de un proceso es $(start, end)$. En muchos sistemas, $start = 0$.

En los SO tipo UNIX, la llamada al sistema `sbrk(n)` permite extender/reducir el segmento de datos del proceso asignando o liberando memoria y actualiza el *program break*.

El subsistema de gestión de procesos se encarga de la creación, cambios de estado, cambios de contexto, sincronización y finalización de procesos. Mas abajo se describe en detalle el uso de los atributos mencionados. También se encarga de mantener el registro de los recursos adquiridos por cada proceso.

Llamadas al sistema de gestión de procesos

Algunas de las llamadas al sistema para la gestión de procesos se describen a continuación.

- `fork()` : Crea un nuevo proceso (hijo) el cual es una copia del proceso invocante. El proceso hijo *hereda* los descriptores de los archivos abiertos del padre.

- `exit(exitstatus)` : Finaliza el proceso.
- `wait(&childstatus)` : Espera (se bloquea) hasta que finalice un proceso hijo.
- `getpid()` : Retorna el identificador del proceso corriente.
- `kill(pid)` : Envía una señal (terminación) al proceso con id `pid` .
- `sleep(n)` : Espera (se bloquea) por `n` segundos.
- `exec(filename, args)` : Reemplaza la imagen de código y datos del proceso corriente por el código y datos de `filename` . Al final de esta sección se describe en mayor detalle.
- `sbrk(n)` : Asigna `n bytes` de memoria extra al proceso invocante.

Estados de un proceso

Un proceso puede estar en uno de los siguientes estados:

- ***RUNNING***: Está ejecutando actualmente, tiene asignada una CPU.
- ***RUNNABLE*** o ***READY***: Está esperando a que el kernel le asigne una CPU.
- ***SLEEPING*** o ***WAITING***: Está *bloqueado* hasta la ocurrencia de un evento que lo despertará.

La **figura 3** muestra los cambios de estado que puede tener un proceso y las funciones de un kernel involucradas en las transiciones.

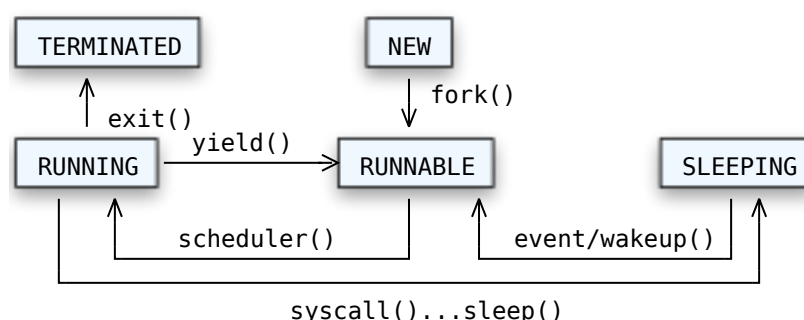


Figura 3: *Cambios de estado de procesos.*

Pueden existir otros estados. Por ejemplo en los sistemas tipo UNIX, un proceso que ha terminado su ejecución (vía `exit()`) pero aún existe en el sistema se denomina ***zombie***. Esto ocurre comúnmente cuando el proceso *padre* aún no ha ejecutado `wait(&status)` y se requiere mantener su estado de terminación en el descriptor del proceso hasta que el proceso padre ejecute el `wait()` correspondiente.

En los sistemas operativos tipo UNIX, cada proceso, excepto `init` (el primer proceso lanzado por el kernel luego del *booting*) tiene un padre. El kernel debe mantener esta invariante.

Nota

¿Qué sucede si un proceso padre termina antes que sus hijos? En UNIX, los procesos hijos son *adoptados* por `init`.

Representación de procesos

Una de las estructuras de datos principales es el conjunto de *process control blocks (PCB)* o *descriptores de procesos*. Este conjunto puede estar implementado como un arreglo (tabla), diccionario o listas.

Un *descriptor* de proceso es una estructura o registro que contiene campos para representar los atributos ya mencionados.

Linux

Los descriptores de procesos están representados en la estructura de datos

Las operaciones que realizan los cambios de estado de un proceso se muestran en la [figura 3](#). No confundir estas funciones con aquellas que implementan las llamadas al sistema con el mismo nombre, como `sleep` o `fork`. Las funciones que implementan los *syscalls* comúnmente invocan a estas funciones.

En los sistemas tipo UNIX, cada proceso (excepto *init*) tiene un padre: El proceso que invocó el syscall `fork` correspondiente. Esto induce a una estructura de árbol como representación de todos los procesos del sistema.

Sobre la tabla de procesos comúnmente también se representan colas o listas para representar sub-conjuntos de procesos. Por ejemplo, un *scheduler* que implementa una política *FIFO*, implementará el conjunto de procesos `RUNNABLE` como una cola, lista o red-black tree.

Un ejemplo interesante es la implementación de [listas doblemente encadenadas intrusivas](#) en GNU-Linux.

mos-steps

`scheduler()` recorre toda la tabla de procesos hasta encontrar un proceso `RUNNABLE`.

Cuando un proceso está en estado `SLEEPING`, el sistema deberá conocer su motivo. Un diseño e implementación común es asociar el conjunto de procesos que esperan al evento que los despertará en una *wait queue*.

Otro aspecto a tener en cuenta es qué sucede si el *scheduler* no encuentra un proceso `RUNNABLE` (situación muy común). Se pueden implementar alguna de las siguientes alternativas:

1. Hacer que `scheduler()` quede en un ciclo hasta encontrar uno.
2. Usar un proceso `idle`, creado inicialmente por el sistema y que ejecute `while (true) yield();`. Este proceso al obtener la CPU la libera inmediatamente. Esto hace que siempre haya un proceso `RUNNABLE`.

En cualquier caso ésto es una oportunidad para *contabilizar* la carga de trabajo (o *idleness*) del sistema para realizar acciones como por ejemplo, ahorro de energía^[1] o disparar operaciones diferidas (como procesar paquetes de red recibidos) en un kernel thread.

Recordemos que el kernel tomará el control cada vez que ocurra un *trap*.

Traps

Un *trap* es una abstracción de los siguientes eventos:

- El proceso ejecutó un *syscall* (instrucción `ecall` en *risc-v*), o
- la CPU disparó una excepción porque el proceso intenta ejecutar una instrucción inválida (privilegiada, división por cero, etc) o porque intenta acceder a un área de memoria inválida, o
- un dispositivo generó una *interrupción*. Un kernel con *preemptive multi-tasking* usa un *timer* o *clock* que genera interrupciones periódicamente.

Otra estructura de datos importante es la representación de la *cpu*, la cual comúnmente contiene comúnmente los siguientes atributos:

- Identificador de la CPU (por si hay más de una)
- Una referencia al proceso o thread corriente en ejecución por esa CPU.
- El contexto salvado del *scheduler* o *idle*: Estado de la CPU del último *cambio de contexto*.
- Estado de las interrupciones (habilitadas/deshabilitadas) y otros.

En un sistema que soporte *múltiples CPUs* (ej: multicore) deberá tener una estructura por CPU.

En el paso `05-tasks` de `mos-steps` se definen algunas de estas estructuras.

Llamadas al sistema

Comúnmente, a cada llamada al sistema se le asigna un identificador único. Es común asignarles números naturales (en xv6, ver `kernel/syscall.h`).

Las llamadas al sistema se implementan en dos partes:

- Invocación desde un proceso (*user mode*).

Generalmente se implementan como funciones en la biblioteca estándar que se enlace con cada programa de usuario. Cada función se implementa comúnmente poniendo el código de *syscall* en un registro designado de la cpu (`a7` en *risc-v*) y luego ejecutar una instrucción de *trap* o *software interrupt* (`int` en x86 o `ecall` en *risc-v*).

- Funciones de servicios en el kernel.

Al ejecutarse una instrucción de *trap* correspondiente a un *syscall*, se dispara el mecanismo de manejo de interrupciones y se ejecuta el *trap handler* correspondiente en el kernel. En este caso, se invocará a un *dispatcher* (`syscall()`) que analiza el código del *syscall* (está en un registro de la cpu) e invoca a la función de servicio correspondiente.

Generalmente las *syscalls* retornan algún valor o código de error a los procesos. En el caso de una API *POSIX*, los valores diferentes de cero son códigos de error.

Los syscalls `exit` y `wait`

Estas dos llamadas al sistema *POSIX* están relacionadas.

- `exit(exitcode)` : Finaliza el proceso corriente con código de salida `exitcode` .
- `wait(&exitcode)` : El proceso corriente espera (se bloquea) hasta que finalice un hijo. El código de salida del hijo se *copia* en el parámetro de salida `exitcode` .

Esto ofrece un mecanismo de sincronización y comunicación básico entre procesos. El proceso padre puede saber cómo terminó el hijo.

Por convención se asume que el código o *status* de salida 0 es *terminación normal* (o exitosa). Un código de salida diferente a cero representa una finalización anormal y el código indica el motivo. Cada aplicación define el significado de los códigos de salida anormales.

Esto motiva el estado **ZOMBIE** de un proceso que ejecutó **exit()** antes que el padre ejecute **wait()**, ya que al menos se debe mantener el *código de salida* hasta que el padre ejecute **wait()**.

En el caso que el proceso padre finalice sin hacer un **wait()**, los sistemas tipo UNIX comúnmente hacen un *reparent* del proceso hijo, comúnmente haciendo que lo *adopt*e **init**.

En un shell el código de salida del último proceso queda en la *variable de ambiente* **\$?**.

Cambios de contexto de procesos

Un hilo de un proceso de usuario puede estar ejecutando en dos modos:

1. *User mode*: La CPU está ejecutando instrucciones del proceso.
2. *Kernel mode*: Se ha alterado el flujo de ejecución normal del proceso debido a la ocurrencia de un **trap**:

El kernel, en su inicialización, definió un *vector de interrupciones*, el cual define un **trap handler** para *atrapar* o manejar los **traps** (interrupciones, excepciones generadas por la CPU e interrupciones por software).

Al ocurrir un **trap**, el hardware y el **trap handler** correspondiente realizan:

1. La CPU pasa a *kernel mode*.
2. Se *salta* al **trap handler** correspondiente.
3. Si el trap se produjo en *user mode*, cambia al *mapa de memoria* del kernel y cambia al *stack en modo kernel* del proceso corriente.
4. Salva el estado de la CPU (valores de *todos los registros*) en el stack en modo kernel. Este estado se denomina **trapframe**.
5. Procesa el **trap** llamando a diferentes funciones del kernel. En el camino puede decidir darle el control a otro proceso (*context switch*), cambiando el *current stack*.
6. Antes del retorno de la interrupción, el **trap handler** recupera el estado de la CPU salvado en el **trapframe** (desde el stack corriente).
7. Retorna del **trap** ejecutando una instrucción de *retorno de interrupción* o *retorno del modo supervisor* (ej: **iret** en x86, **sret** en risc-v). Luego del retorno del kernel el thread del nuevo proceso seleccionado continuará su ejecución desde el punto en que había sido interrumpido en el pasado.

En el caso el *trap* ocurra en *user mode*, el uso de una pila diferente impide que un proceso de usuario pueda *filtrar* datos en la pila dejados por el código del kernel, lo cual sería un problema serio de seguridad. Un proceso ejecutando en modo usuario no tiene acceso al *kernel mode stack*.

El kernel decide que el proceso corriente debe abandonar la CPU en base a:

1. El proceso completó su *quantum* o *time slice* de uso de CPU. Esto se produce cuando el kernel recibe un *timer interrupt*. En este caso el kernel invoca a una función del tipo *yield()*, la cual cambia al contexto del *scheduler*. Esto es típico de un SO con *preemptive multitasking*.
2. El proceso realizó una llamada al sistema, como *read()*, *write()*, *sleep()* u otra, cuyo procesamiento puede llevar demasiado tiempo por lo que conviene suspender el proceso (pasar al estado *SLEEPING*). El kernel invoca a una función del tipo *sleep(wait_queue)* e invoca a *yield()* para que éste asigne la CPU a otro proceso.

Un proceso *sleeping* se *despertará* (pasará a *RUNNABLE*) cuando ocurra un evento como por ejemplo, la finalización de la operación de entrada-salida por la que está esperando.

Ejecución de programas de usuario

Un proceso en modo usuario se representa como una *task* en el kernel. La única diferencia es que cada proceso de usuario tiene, además del stack en modo kernel, un stack para su ejecución en modo usuario.

1. Busca e inicializa un descriptor de tarea.
2. Invoca a *exec("program")* el cual asigna memoria, carga las secciones de ejecutable desde el disco, arma la tabla de páginas y configura el *trapframe* en el stack en modo kernel. Además en la pila en modo usuario apila los valores de los argumentos de *main*.
3. Se configura el *contexto* de la tarea vía *init_context(userret, task->sp)*. La función *userret()* configura la CPU para luego retornar desde un trap (vía *trapret*) a modo usuario. En RISC-V, esto requiere que antes de retornar a modo usuario debamos hacer:
 - Setear el modo previo de ejecución a *user mode*: *sstatus.SPP = 0*,
 - Habilitar interrupciones en modo usuario: *sstatus.SPIE = 1*,
 - Definir el retorno al inicio o punto de interrupción previo del programa: *sepc = trapframe->ra*.

Luego, `scheduler()` al encontrar a este proceso como `RUNNABLE` hará el *context switch* saltando a un camino de *retorno de un trap en modo supervisor* hacia modo usuario. Más abajo se describe `exec` en forma mas detallada.

Cuando un proceso de usuario esté en ejecución, éste podrá ser interrumpido (por un dispositivo, excepción de la CPU o hace una llamada al sistema).

En éste punto el *trap handler* del kernel deberá cambiar del stack en modo usuario al stack en modo kernel, en donde se apilarán los valores de los registros de la CPU salvados (*trapframe*).

El primer proceso de un sistema

En los sistemas tipo UNIX, el primer proceso lanzado es `init`.

El proceso `init` generalmente se vincula a las *terminales* y dispara el proceso `login` en cada una. Finalmente `login` creará un *shell* para que un usuario comience su interacción con el sistema. La siguiente figura muestra el árbol de procesos inicial en un sistema tipo UNIX.

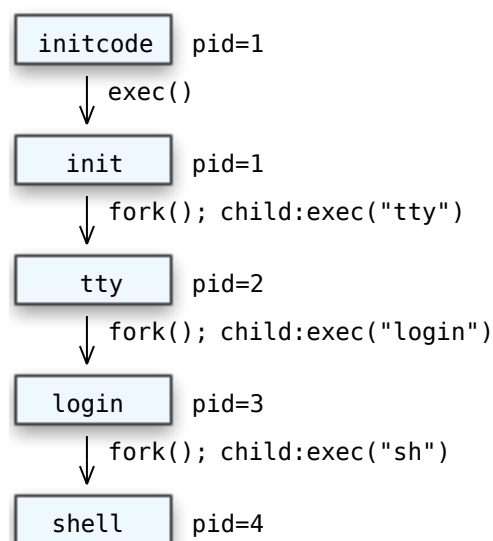


Figura 5: Inicio del sistema en sistemas UNIX.

En `ed0s`, haremos que `init` simplemente ejecute el shell (haciendo `fork()` y en el hijo `exec("shell", [])`).

La llamada al sistema `exec`

El syscall `exec(cmd, args)` realiza básicamente los siguientes pasos:

1. Reserva memoria para el código, datos y *user-mode* stack y carga el código y los datos del programa (ejecutable) `cmd` en la memoria asignada. Al mismo tiempo, construye

- la tabla de páginas para implementar protección y *memoria virtual*.
- Configura la pila en modo usuario (apilando los valores de los argumentos de `main`).
- Configura el *trapframe* en la pila en modo kernel. En `trapframe->ra` (valor del `pc` salvado en la interrupción previa) será el *punto de entrada del programa* (dirección de memoria *entry* en el archivo ELF). El valor de `trapframe->sp` será el del tope de la pila en modo usuario.
- Libera la imagen de memoria anterior del proceso.
- Configura la cpu con el nuevo *mapa de memoria* creado en el paso 1 (en RISC-V, `task->vm` apunta a la tabla de páginas creada).

En *xv6* esta función está implementada en `exec.c` .

El proceso, luego al ser planificado, retornará de modo *kernel* a modo *usuario* ejecutando la función correspondiente al punto de entrada (comúnmente `main()` o `start()`) del programa.

La llamada al sistema `fork()`

Esta llamada al sistema crea un nuevo proceso (*hijo*) que es una copia o *clon* del proceso invocante (*padre*).

Básicamente realiza los siguientes pasos:

- Obtiene un nuevo descriptor para el nuevo proceso (*hijo*) y un nuevo *pid*.
- Copia el `contexto` del padre en el descriptor del hijo. Ambos hilos de ejecución deberán continuar en el mismo estado.
- Asigna memoria y copia los bloques de memoria de código, datos estáticos y stack en modo usuario. En el proceso crea el *mapa de memoria correspondiente* (tabla de páginas).
- En el stack en modo kernel del hijo setea el valor de retorno del syscall en cero (en RISC-V, `trapframe->a0 = 0`). En el trapframe del padre pone como valor de retorno el pid del hijo (en RISC-V, `trapframe->a0=tasks[child].pid`).
- Setea el estado del proceso hijo en `RUNNABLE` .

En cada proceso, al retornar del modo supervisor, éstos continuarán en el punto de retorno de `fork()` . En el padre retornará el *pid* del hijo y en el hijo 0.

 El ahorro de energía puede apagar o suspender subsistemas y bajar la frecuencia de operación de la cpu.

Herramientas de desarrollo **Planificación de uso de cpu**